

Exercise 3: Architect First: Plan Mode and the Explore Agent

DEV CYCLE	Design (architecture)
CC FEATURES	Plan mode, /effort, the Explore subagent, @file references, reviewing a plan before code
SPEC TRACES	All of SPEC.md; produces docs/ARCHITECTURE.md and spec change requests
TIME	30 min
YOU PRODUCE	An agreed architecture in docs/ARCHITECTURE.md and a list of spec gaps resolved before coding

Objective

The single biggest difference between amateur and professional AI-assisted development is planning before building. In a spec-driven project the plan has a second job: it tests the spec. You will use plan mode to have Claude turn SPEC.md into an architecture, make it find the holes in the spec, decide those holes yourself, and only then write anything to disk. Nothing is built here, and that is the point.

Plan mode is cheap. Reviewing a plan takes 30 seconds; reverting wrong edits takes 30 minutes. Force a plan for anything that touches more than two files, every time.

Walk-through

1 Enter plan mode and set the effort

Press Shift+Tab until the mode indicator reads plan. In this mode Claude cannot edit files; it can only read, think and propose. Then run `/effort` to see and adjust how long Claude reasons: low is faster and cheaper, higher levels reason longer on hard problems. Architecture is a good moment to raise it.

INSIDE THE CLAUDE CODE SESSION

```
> /effort
```

VALIDATE The mode line shows plan mode. Ask it to create a file: it declines and offers a plan instead.

2 Brief the architecture from the spec

Ask for a complete design grounded in the spec, and make Claude map every requirement to a place in the code. Be demanding now, while it is cheap.

TYPE THIS PROMPT INTO CLAUDE CODE

Read @docs/BRIEF.md, @docs/SPEC.md and @docs/POLICY.md. Design the architecture for DischargeFlow. Produce:

- 1) a module map: each file in app/ and what it owns, and which requirement IDs (FR/AI/SEC) it implements;
- 2) the draft-generation sequence step by step, from POST /discharges/{id}/drafts to a stored draft, showing where retrieval, PHI masking, the Claude call, validation and safety flags happen;
- 3) the trust boundaries: which data is untrusted, which data leaves the process, and where masking sits;
- 4) a list of ambiguities or gaps in SPEC.md, phrased as questions I must decide.

Do not write code or files yet.

VALIDATE You get a module map that covers db, models, schemas, routes, drafting, rag, sanitize, safety and agent, with requirement IDs against each; a sequence in which masking happens before the Claude call and the medication table is rendered from the DB after it; and at least three genuine spec questions.

SCREENSHOT SLOT *Plan-mode architecture with module map and spec questions*

3 Stress-test the plan

Do not accept the first plan. Push on the weakest points. Your job as the architect is asking the right questions, not typing every line.

TYPE THIS PROMPT INTO CLAUDE CODE

Three challenges. First: two reviewers click approve on the same draft within a second of each other; what stops a double decision, at the database level and not just in Python? Second: the model mentions ibuprofen, which is not in the record; where exactly is that caught and what does the reviewer see? Third: the Claude API times out after 30 seconds; what does the clinician get back, and what state are the record and draft left in? Revise the plan to handle all three and cite the requirement IDs.

VALIDATE The revised plan uses a unique constraint on `reviews.draft_id` (or a conditional update on status) so a second approve gets 409 (FR-11); a post-generation medication cross-check adds `unknown_medication:ibuprofen` to `safety_flags` (AI-04); and a timeout stores a `generation_failed` draft, leaves the record status unchanged and returns 503 (FR-06). If it hand-waves any of these, push again.

4 Close the spec gaps before you build

Answer the spec questions from step 2 yourself; you are the product owner. Then have Claude write your decisions into the spec with a changelog, so the decision lives in the contract rather than in this chat. This is change control, and it is what keeps a spec-driven project honest.

TYPE THIS PROMPT INTO CLAUDE CODE

Here are my decisions on your spec questions: [write your answers]. Add a "## 14. Changelog" section to docs/SPEC.md with today's date and one line per decision, and update the affected requirement text. Show me the diff before applying.

VALIDATE Leave plan mode first (Shift+Tab to normal). Claude shows the diff to SPEC.md and asks permission. Every change is small, dated, and tied to a requirement ID. If a "decision" quietly changes something you did not decide, reject the edit.

5 Capture the architecture to disk

Now have Claude write the agreed design into a docs file. This becomes your reference for the build and your handoff artefact later.

TYPE THIS PROMPT INTO CLAUDE CODE

Write the final agreed architecture to docs/ARCHITECTURE.md: module map with requirement IDs, the draft-generation sequence as a Mermaid sequence diagram, the trust-boundary section, the record and draft state machines from SPEC §5.2, and the decisions from the stress test.

VALIDATE Claude asks permission, then creates docs/ARCHITECTURE.md. Read it end to end. If you disagree with anything, fix it now, not after 500 lines of code exist.

SCREENSHOT SLOT *docs/ARCHITECTURE.md open in VS Code with the Mermaid sequence diagram*

What success looks like

- A written, reviewed architecture exists before any application code, and every module lists the requirement IDs it owns.
- You forced at least three design decisions (race on approve, unknown medication, AI timeout) instead of letting Claude pick silently.
- Spec gaps you found are resolved in docs/SPEC.md with a changelog entry, not left in chat history.
- You can explain why masking must happen before the Claude call and the medication table must be rendered after it.

Common pitfalls (Windows and macOS)

- Skipping plan mode and saying "build the API" makes Claude take a dozen silent architecture decisions you will spend Exercise 8 unwinding.
- @docs/SPEC.md is a file reference: the @ pulls the actual file into context. A typo in the path means Claude guesses.
- Editing the spec to match whatever Claude proposed is not change control. You decide, then the spec changes.

Go deeper · AI developer track

Optional, for faster finishers. Each challenge extends the lab with a production-grade technique.

- In plan mode, have Claude draft docs/adr/0000-sqlite-bm25-over-postgres-pgvector.md: context, options, decision and the exact conditions under which you would reverse it.
- Ask for the planned module import graph and check it for cycles (for example, routes → drafting → routes). Fix the layering on paper before any file exists.

Stretch goal

Ask Claude to use the Explore agent to scan the repo read-only and check two things: that the planned layout does not clash with anything that exists, and that every file in `samples/records/` conforms to the record schema in SPEC §5.1. On a real codebase, this read-only pass is how you onboard safely.